

Recap — Local Summariser Manual

recap / recaplong / recapall / recapurl / recapurls / recapc / recapmail / recapsetup / recapdoctor · MacBook Air M1, 8 GB · Ollama + Gemma 4 · functions in recap.zsh, loaded from ~/.zshrc · version 3.5, 6 Oct 2026

Read this first. Recap runs on macOS only (it uses pbcopy, pbpaste and zsh) and has been tested on one MacBook Air M1 with 8 GB of RAM. Summarising always happens on your Mac; copied text and PDF files on your Mac never leave it. recapurl and recapurls download the page, captions or PDF you ask for directly from its site. Everything Recap saves, in ~/Summaries, is plain unencrypted text. Summaries come from a small model and can be wrong: check the source before relying on one, and do not use a summary alone for a legal, medical or financial decision. Provided as is, under the MIT licence. Run recapdoctor at any time to check the setup.

1. What is set up

Item	Name / location	Purpose
Functions	recap.zsh, loaded by one source line in ~/.zshrc — recap, recaplong, recapall, recapurl, recapurls, recapc, recapmail	Read the clipboard or a web address, summarise locally, save Markdown
Summaries folder	~/Summaries/YYYY-MM-DD-title.md	Every run saves a Markdown file here automatically. Address runs also save the text the model read (-source.txt); long texts save -notes.txt
gemma4-sum (default)	~/Modelfile.g4 — FROM gemma4:e2b, num_ctx 12288, temperature 0.2	Current summary model. Runs 100% on GPU
gemma-sum (optional)	~/Modelfile.sum — FROM gemma3:4b, num_ctx 12288, temperature 0.2	Older Gemma 3 model. Useful as a second opinion
llama3.2:3b	Base model	Used by recapmail
Python packages	trafilatura, youtube-transcript-api, pypdfium2, in a private environment at ~/.recap/venv	Needed only by recapurl and recapurls (section 4). Installed by recapsetup. pypdf is an optional fallback for PDFs

Nothing updates itself. `ollama pull gemma4:e2b` refreshes the base model; afterwards rebuild with `ollama create` (section 6). Summarising always runs on this Mac. recap, recaplong, recapall and recapmail work offline; recapurl and recapurls need a network connection to fetch a page, a YouTube video's captions or a PDF at an address (a PDF file already on the Mac is read offline).

2. Daily use

Copy the text (Cmd+C), open Terminal, type the command. The first run after a break is slower because the model has to load. Each summary prints on screen and is saved as Markdown in ~/Summaries, with the article's headline as the first line.

Command	What it does
recap	Adaptive summary: heading, gist, main points, and "Worth noting" only if there is more
recap short	One sentence plus up to 3 takeaways
recaplong	Long articles and transcripts. Reads the text in parts, then combines. Slow but far more accurate
recap changes	For circulars and rule changes. Names which clauses are new and which are existing, who must act, and from when
recap "focus on costs"	Your own focus. Also useful: recap "this is an explainer, not an editorial"
recapc	Same as recap, and also copies the summary to the clipboard
recapall	Several articles in one go (section 3)
recapurl "address"	A web article, YouTube video or PDF (an address or a file on the Mac), fetched and summarised (section 4)

Command	What it does
recapurl "A" "B"	Several addresses or PDF files one after another. --single gives one combined file, --separate one file each
recapurls	Every web address on the clipboard, one per line (section 4)
recapsetup	Creates or updates the private Python environment that recapurl and recapurls use (section 4)
recapdoctor	Checks the whole setup and says what to fix. Safe to paste into a bug report
recapmail	3-line email summary: sender, tasks, deadlines, reply needed
RECAP_MODEL=gemma-sum recap	Use a different model for this one run only
recap help	Lists the options

The first line, “Reading N words...”, confirms what was captured. If the count looks wrong, copy again. To summarise a file instead of copying: `pbcopy < ~/file.txt` then `recap`.

3. Several articles in one go (recapall)

In a Word file, paste the first article, then on a line of its own type four @ signs (@@@@), then the next article, and so on. Use that marker, not --- or ###, because Word turns those into a horizontal line. Press Cmd+A, Cmd+C, then run `recapall`. Each article is summarised separately. An article over 4,000 words is read in parts automatically, the way `recaplong` does it (about 45 seconds per 1,200 words), so a long article in the batch no longer risks a reversed meaning; change the limit with `RECAP_LONG_WORDS`. Allow 30 to 90 seconds per normal article and keep the Mac awake and plugged in. Start each article with its headline, because the first line becomes the heading.

At the end every summary is on the clipboard and in one saved Markdown file, so **copy the articles again before the next run**. If no article has at least 30 words, `recapall` saves nothing and leaves the clipboard alone.

4. Web pages, YouTube and PDFs (recapurl, recapurls)

Instead of copying the text, give the address. `recapurl` fetches the page, the captions of a YouTube video, or a PDF (an address, or a file on the Mac), and then summarises it exactly as `recap` does.

```
recapurl "https://example.com/article"
recapurl "https://www.youtube.com/watch?v=VIDEOID"
recapurl "https://example.com/paper.pdf"
recapurl ~/Downloads/paper.pdf
recapurl "https://example.com/article" "focus on costs"
recapurl --single "https://example.com/one" "https://example.com/two"
recapurls                # every address on the clipboard, one per line
```

Always put the address in double quotes. Addresses containing ? or & break otherwise. With several addresses, the extra ones must start with `https://` (or `www.`) or be a `.pdf` file. A mode (short, changes) or focus text placed after them applies to every address. On a page over 4,000 words, short and changes are ignored with a note.

Link	What happens
Web article	Fetches the page, strips menus, ads and comments, adds the headline and byline, then summarises. Over 4,000 words it is read in parts, like <code>recaplong</code> .
YouTube video	Reads the captions: English uploaded by the channel first, then YouTube's automatic English, then any language. Adds the video title and “Channel:”, which is the uploader, not necessarily the speaker.
PDF	An address, or the path of a file on the Mac (a PDF served without <code>.pdf</code> in its name is detected too). The text is cleaned first: line breaks joined into paragraphs, words split by a hyphen mended, and page numbers, repeated headers and footers, page-edge web addresses and copyright lines dropped. The title is the first short line of page 1, else the file name.
Channel or playlist page, Vimeo, Spotify, audio or video file	Refused with a message. A video without captions is also refused.

Files. Each run saves `DATE-title.md` with a Source: line, marked “(YouTube captions, automatic)” or “(text appears cut off: possibly a paywalled teaser)” where that applies. A PDF is marked “(PDF)”. It also saves `DATE-title-source.txt`, the exact text the model read, so a summary can be checked against it. Long texts also

save -notes.txt.

Several addresses run one after another under “Link 2 of 4”. A failed address is skipped, and the final list shows each failure with its reason. Where the summaries go:

Choice	How to ask for it
One combined file	--single, or answer o when asked, or RECAP_BATCH=single
One file per address	--separate, or answer s or Enter when asked, or RECAP_BATCH=separate

The order is: flag, then RECAP_BATCH, then a one-key question (asked once, only in a terminal; if it cannot ask, separate files are used). With one address nothing is asked. The combined file is ~/Summaries/DATE-batch-of-N-Links.md: it starts with “# N link summaries, DATE”, separates the summaries with ---, and ends with a “Not summarised” section giving each reason. In that mode no individual summary files are created; the source files are still saved and the clipboard is not touched.

Length limit. Above 10,000 words recapurl stops (web page, YouTube transcript or PDF), saves the text, and suggests copying one chapter or section and using recaplong. The part-by-part method has been tested only up to about 6,600 words, and past roughly 10,000 words its combined notes probably no longer fit in the model's memory. RECAP_URL_MAX=20000 recapurl "address" raises the limit for one run.

It stops instead of summarising a paywall notice or an advert in the common cases (section 8 lists each message). The check is not perfect, so read the “Text starts” lines it prints. **YouTube:** automatic captions have no speaker names, little punctuation and some misheard words, so check names, figures and who said what against the video. youtube-transcript-api is unofficial and breaks when YouTube changes; if it fails, run recapsetup again to upgrade it. **PDF layout:** two-column pages, tables and footnotes can come out jumbled, and scanned PDFs have no text to read; look at the saved -source.txt when the layout is complicated. **Install once:** run recapsetup. It creates a private Python environment in ~/.recap/venv and installs the three packages there (about a minute, needs a network connection). It never touches your own Python, so it also works when pip refuses to install into a Homebrew-managed Python (the externally-managed-environment error). Run it again to upgrade the packages. Recap uses RECAP_PYTHON if set, then the private environment, then python3 from the path. Then run recapdoctor.

5. Length — the real limits

The context window (num_ctx 12288) holds roughly 7,000 words, but **accuracy falls long before that ceiling**. The model keeps producing confident output as quality degrades, so length is the single biggest risk to trust.

Article length	What to expect	What to run
Up to ~1,500 words	Reliable. Figures, names and attribution generally correct	recap
1,500 – 4,500 words	Mostly good (a 4,400-word essay checked clean); occasional dropped detail	recap, spot-check key figures
Above ~5,000 words	Meaning can reverse in a single pass	recaplong (recapall, recapurl and recapurls do this automatically above 4,000; recapurl stops above 10,000)

One real failure found in testing. In a 5,900-word essay, “without solitude there would be no America” became “independent thinking without solitude” — the argument reversed. The text fitted inside the context window, so length alone was the likely cause. This came from a single run, so treat the thresholds above as guidance, not a measured limit.

How recaplong works: it splits the text into roughly 1,200-word parts, takes detailed notes on each part separately, then writes the summary from the notes. On a 6,600-word transcript every claim checked was accurate, but its “Worth noting” list repeated points. A filter now drops “Worth noting” bullets that mostly repeat the main points (RECAP_OVERLAP, default 0.6; lower it to filter more), and it worked on the same transcript. Expect about 45 seconds per part, so a 6,000-word essay takes four to five minutes, and the summary appears after the progress lines, not word by word. The intermediate notes are saved next to the summary as a .txt file, useful when a claim needs checking. Change the part size with RECAP_CHUNK=900 recaplong.

6. Model, temperature and context

```
open -e ~/Modelfile.g4          # edit temperature or num_ctx, then save
ollama create gemma4-sum -f ~/Modelfile.g4 # rebuild, seconds, no new download
ollama show gemma4-sum          # check the settings took effect
ollama stop gemma4-sum          # unload the old copy
```

Task	Temperature
Summaries, facts, circulars, coding	0.1 – 0.3 (current: 0.2)
General questions	0.5 – 0.7
Drafting, ideas, creative writing	0.8 – 1.0

Building from scratch:

```
ollama pull gemma4:e2b
printf 'FROM gemma4:e2b\nPARAMETER num_ctx 12288\nPARAMETER temperature 0.2\n' > ~/Modelfile.g4
ollama create gemma4-sum -f ~/Modelfile.g4
```

Context length: gemma4:e2b supports 128K tokens, but num_ctx is capped at 12288 because the memory it needs has to fit in 8 GB. Raising it is possible — try 16384 and watch ollama ps — but it does not fix the accuracy fall described in section 5, so recaplong remains the better answer for long text. **Check memory** while a summary runs by opening a second Terminal window and typing ollama ps. The PROCESSOR column should read 100% GPU; a CPU/GPU split means the model no longer fits and will be slow.

Thinking mode: Gemma 4 and Qwen print their reasoning before the answer. The functions suppress this with --think=false, added automatically for models whose name starts with gemma4 or qwen. Gemma 3 does not accept the flag, which is why the rule is conditional.

Alternatives tried (one three-article batch each, 5 Oct 2026): phi4-mini copied the example sentence from the prompt into its output and skipped the gist. qwen3.5:4b did not fit fully on the GPU (23%/77% CPU/GPU) and invented one figure. gemma4-sum stays the default. One run each is guidance, not a measured result.

Settings that can be put before a command, for example RECAP_BATCH=single recapurl, or exported in ~/.zshrc:

Setting	Default	What it changes
RECAP_MODEL	gemma4-sum	Model for one run
RECAP_DIR	~/Summaries	Output folder
RECAP_CHUNK	1200	Words per part when reading long text
RECAP_LONG_WORDS	4000	Above this, recapall and recapurl read in parts
RECAP_OVERLAP	0.6	How much a “Worth noting” bullet may repeat the main points before it is dropped
RECAP_BATCH	(ask)	single or separate, for several addresses
RECAP_URL_MIN	150 (60 YouTube, PDF)	Fewest words recapurl accepts
RECAP_URL_MAX	10000	Most words recapurl accepts
RECAP_PDF_MAX_MB	30	Largest PDF recapurl downloads or opens
RECAP_PDF_READER	(pypdfium2)	pypdf forces the fallback reader, for troubleshooting
RECAP_HOME	~/.recap	Where recapsetup puts the private Python environment
RECAP_PYTHON	(none)	A Python to use instead of the private environment
RECAP_BASE_PYTHON	python3	The Python recapsetup uses to create the environment
RECAP_URL_FORCE	off	1 skips the paywall, short-text and length checks

7. Changing or re-installing a function

The functions live in recap.zsh. ~/.zshrc holds one line, source /path/to/recap/recap.zsh. To change them, replace that whole file with the new one. Do not paste the functions into ~/.zshrc as well: two copies make the old behaviour come back.

```
cd /path/to/recap          # the folder that holds recap.zsh
zsh -n recap.zsh           # syntax check: no output means OK
source ~/.zshrc            # load the change into this Terminal window
grep -c "^recap()" recap.zsh # must print 1
grep -c "recap.zsh" ~/.zshrc # must print 1 (the single source line)
```

Keep `alias recap='recap | tee >(pbcopy)'` exactly as written — no space between `>` and `(pbcopy)`. The functions already strip stray “Here’s a summary...” openings and trailing “Related:” link lists, convert bullets to Markdown, and bold the “Worth noting” heading. Commit `recap.zsh` to git, never the whole `~/.zshrc`.

8. Troubleshooting

Problem	Cause and fix
“Only 3 words on the clipboard”	Clipboard did not hold the text. Copy again, or <code>pbcopy < ~/file.txt</code>
command not found: recap	Run <code>source ~/.zshrc</code> , or open a new Terminal window
Old behaviour after an edit	Forgot <code>source ~/.zshrc</code> , or two copies of a function exist (see the grep checks in section 7)
“Thinking...” block appears	<code>--think=false</code> missing, or the model name does not start with <code>gemma4/qwen</code>
Heading is wrong or blank	The first line of the copied text is not the headline — copy from the headline down
Summary reverses or garbles a long text	Text too long for one pass. Use <code>recaplong</code>
<code>zsh: unknown file attribute: b</code>	A space was typed in <code>>(pbcopy)</code> ; it must be <code>>(pbcopy)</code>
Mac slow, fan loud	<code>ollama ps</code> , then <code>ollama stop MODELNAME</code> ; quit heavy apps
recapall: “Only one article found”	The clipboard no longer holds your articles (recapall replaces it with the summaries) or the <code>@@@@</code> lines are missing. Copy the articles again
<code>zsh: no matches found</code> (with an address)	Put the address in double quotes
“trafilatura (or youtube-transcript-api) is not installed”	Run <code>recapsetup</code>
“No PDF reader is installed”	Run <code>recapsetup</code>
“Note: this used the basic PDF reader (pypdf)”	<code>pypdfium2</code> is not installed, so the fallback was used. Headers and small capitals can come out garbled. Run <code>recapsetup</code>
“No readable text in this PDF”	Scanned pages (images). <code>recapurl</code> cannot read those. Use a text version of the document
“This PDF is password-protected”	Remove the password in a PDF viewer, save a copy, and run <code>recapurl</code> on that
“That link did not return a PDF”	The address is a web page or a login page. Open it in the browser and download the real file
“Could not read the PDF (ErrorName)”	The file is damaged. Download it again
“That PDF is larger than 30 MB”	<code>RECAP_PDF_MAX_MB=60</code> raises the limit for one run
“File not found”	Check the path. Quote it if it contains spaces
“That is N words: too long”	Copy one chapter or section from the saved <code>-source.txt</code> and use <code>recaplong</code> , or <code>RECAP_URL_MAX=20000</code> for one run
“The page marks its article as paywalled”	Members-only page. The extracted text is saved so you can look. <code>RECAP_URL_FORCE=1 recapurl "address"</code> summarises it anyway
“Only N words came back”	A paywall notice, login page or script-built page (under 150 words; 60 for YouTube). Look at the saved <code>-source.txt</code> . <code>RECAP_URL_MIN=80</code> changes the limit
“HTTP 403” (also 401, 429)	The site refuses automated downloads or needs a login. Copy the text and use <code>recap</code>
“HTTP 404” or 410	The address is wrong or the page was removed
“Redirected to a login or subscribe page”	Sign-in needed. Copy the text and use <code>recap</code>
“No article text found”	The page is built from scripts. Copy the text and use <code>recap</code>
Source line says “text appears cut off”	Probably a paywalled teaser: the summary covers only the free part
“This video has no captions available”	The owner turned captions off. It cannot be summarised
“YouTube blocked the request”	Try later or on another network, or open the video, choose Show transcript, copy the text and use <code>recap</code>

Problem	Cause and fix
"Could not read the captions (ErrorName)"	YouTube changed something. Run <code>recapsetup</code> to upgrade <code>youtube-transcript-api</code>
"Python 3 was not found" or "Python is too old"	<code>recapsetup</code> needs Python 3.9 or newer. Install one (<code>python.org</code> or <code>brew install python</code>), then <code>RECAP_BASE_PYTHON=/path/to/python3 recapsetup</code>
"Could not create the environment"	That Python has no <code>venv</code> module. Try another with <code>RECAP_BASE_PYTHON=/path/to/python3 recapsetup</code>
"The package install failed"	Usually no network connection, or a Python version the packages do not support. Read the lines shown, then run <code>recapsetup</code> again
<code>recapdoctor</code> reports a PROBLEM	Each line says what to fix. Python problems affect only <code>recapurl</code> and <code>recapurls</code>

9. What this model gets wrong — check these

Tested across circulars, editorials, newsletters, match reports, explainers, long essays, story collections, web pages, YouTube captions and PDFs. No invented facts were found in the summaries checked, but five faults repeat (the first five rows). The next two rows apply to `recapurl` and `recapurls`, and the last two to the PDF and gist problems. Treat a summary as a map of the article, never as the final word, and do not use one alone for a legal, medical or financial decision.

Weakness	What to do
Dropped hedges. "Very difficult", "possibly", "the best explanation is" get trimmed, which makes claims sound firmer than they are	Re-read the source sentence before quoting a strong claim
Who said what. A lesson the author drew is credited to the person the author quotes (seen repeatedly in testing)	Verify every "according to X" before repeating it
New versus unchanged rules. In circulars, existing clauses can be listed as if they were changes	Use <code>recap</code> changes, then read the clauses it names yourself
Flattened names. "A journalist", "a report" in place of the named person or publication	Go back to the article for names you need
Figures. A number from one scenario can migrate to another	Check any figure you will act on
Web extraction. The text that comes back can be a teaser, a paywall notice or an advert, and the summary describes it confidently	Read the "Text starts" lines and the saved <code>-source.txt</code> . The stop messages catch the common cases, not all
YouTube automatic captions. No speaker names, little punctuation, misheard words; attribution is the weakest point	Check names, figures and who said what against the video
PDF layout. Two-column pages, tables and footnotes can come out jumbled or flattened, and the summary describes the jumble	Read the saved <code>-source.txt</code> first when the layout is complicated
Missing gist. Sometimes the summary starts straight with bullets and has no gist sentence (the cause is the model, not the script)	Run it again; the gist came back when this was repeated

Strongest on: news and match reports, explainers, research write-ups, market and business pieces. **Weakest on:** collections of quotes or anecdotes, where attribution slips most, and regulatory texts, where new and unchanged clauses blur. For an important document, run it a second time with `RECAP_MODEL=gemma-sum recap` and compare — where two different models agree, the summary is usually sound.